

M17 / PROTECTED ANSWERS

M17 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

1. DE17-01 | From requirements to architecture 3

2. DE17-02 | Batch vs streaming decisions 3

3. DE17-03 | Warehouse vs lakehouse 4

4. DE17-04 | Managed service vs build yourself 4

5. DE17-05 | Scaling and bottleneck reasoning 5

6. DE17-06 | Reliability and failure domains 5

7. DE17-07 | Cost-aware architecture 6

8. DE17-08 | Security and governance by design 6

9. DE17-09 | Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes 7

10. DE17-10 | Architecture review and defense 7

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after saving the closed-book attempt. These answers are examples, not the only valid architectures.

1. DE17-01 | From requirements to architecture

One acceptable solved answer. A retailer wants hourly store sales by 07:00 local time, can tolerate 30 minutes of delay during recovery, must keep cardholder data out of analytics, and has a two-person data team. Translate this into: batch ingestion every 15 minutes or hourly, curated sales grain, PII tokenization before analytics, a published freshness SLO, replayable raw evidence, simple managed components, and an alert if reconciliation or freshness fails. Notice that the requirements already rule out some unnecessarily complex designs.

Reasoning. Start with six questions: who consumes the data, what decision/action depends on it, how fresh it must be, how wrong it may be before harm occurs, what data is sensitive, and what failure/recovery window is acceptable. Then add volume and growth. Only after these are explicit should you compare architecture options. Separate facts from guesses: '20 million events/day today' is evidence; 'will probably double' is an assumption that should be tested with a scenario rather than treated as truth.

Common mistake. A team selects Kafka, Spark, Kubernetes and a lakehouse because these technologies appear in job descriptions. The workload is one 2 GB nightly file with next-morning reporting. The result creates 24x7 operating cost and a steep on-call burden without a requirement that needs streaming or cluster orchestration.

Validation. An architecture decision sheet linking each important requirement to a design response and a measurable proof. Another engineer should be able to disagree with your choice without first guessing your assumptions.

Changed retry. Change the requirement from 'next morning' to 'fraud alert within 10 seconds' and explain exactly which architecture boundaries must change and which can remain batch.

2. DE17-02 | Batch vs streaming decisions

One acceptable solved answer. Compare three workloads. Payroll: monthly, correctness dominates latency - batch. Card fraud: suspicious transaction must be acted on in seconds - streaming event path plus durable replay. Executive daily revenue: source events may arrive continuously, but a governed hourly/daily aggregate can remain batch or micro-batch. The same company can rationally use all three.

Reasoning. Use a decision tree instead of fashion. Ask: what is the maximum useful delay? Is the source naturally event-driven? Must individual events trigger actions, or are periodic aggregates enough? Can downstream consumers handle continuous updates? What replay and state semantics are required? A 5-minute dashboard might still be micro-batch. A stream can also feed a durable batch/lakehouse path for reconciliation and reprocessing.

Common mistake. A product team asks for 'real time' because the phrase sounds modern. No one can identify a user action that changes within an hour. The stream becomes a permanent

on-call system while reports are opened once each morning.

Validation. A latency/value table and a justified choice that names state, replay, ordering and operational implications - not merely 'streaming is faster'.

Changed retry. Take one batch choice and tighten its useful latency by 100x. Re-evaluate source, storage, compute and monitoring without automatically assuming every component must become streaming.

3. DE17-03 | Warehouse vs lakehouse

One acceptable solved answer. A finance reporting team needs highly governed SQL, stable semantic models and hundreds of BI queries. A data science team needs raw/semi-structured clickstream and Spark. One defensible architecture lands durable source data in object storage/lakehouse tables, curates reusable silver data, and serves governed finance marts through a warehouse/SQL serving surface. The design explains the boundary instead of forcing one label everywhere.

Reasoning. Compare on workload and operating model: BI concurrency, semi/unstructured data, open engine access, ML/Spark needs, governance, cost model, performance predictability and team skills. Do not treat 'cheap storage' as 'cheap system': query engines, compaction, cataloging and operational labor matter. Likewise, do not assume a warehouse cannot ingest files or a lakehouse cannot serve SQL; modern products overlap.

Common mistake. A team migrates all BI tables into a lake because storage is cheaper but does not plan compaction, catalog permissions, concurrency or semantic serving. File cost falls while analyst experience and operations degrade.

Validation. A comparison matrix that connects consumers and data shapes to a storage/serving choice and states what will remain intentionally outside that platform.

Changed retry. Remove Spark/ML from the scenario and make 95% of demand governed BI. Re-evaluate whether the lakehouse remains central or becomes a landing/archive component.

4. DE17-04 | Managed service vs build yourself

One acceptable solved answer. A six-person analytics team needs orchestration. Self-hosting a scheduler on Kubernetes gives control but adds cluster/scheduler upgrades, HA, secrets, logging and on-call. A managed orchestration service may cost more per visible unit yet reduce total ownership. If the company already has a platform team operating Kubernetes safely, the balance may change.

Reasoning. Score choices on required control, compliance, portability, team expertise, uptime responsibility, upgrade burden, integration, cost and exit plan. 'Managed is expensive' ignores the labor and risk of self-operation. 'Managed is easy' ignores quotas, platform constraints and provider-specific semantics. The best choice often manages commodity layers and keeps business-differentiating logic portable.

Common mistake. The team self-hosts because the open-source software is free. Six months later one engineer spends most of the week on upgrades and broken credentials. License cost was zero; total cost of ownership was not.

Validation. A weighted scorecard with assumptions and an explicit owner for the operational responsibilities that remain after the choice.

Changed retry. Change the organization to a regulated platform team with 30 experienced SREs and a hard requirement for custom network/security controls. Re-score rather than keeping your first answer.

5. DE17-05 | Scaling and bottleneck reasoning

One acceptable solved answer. A pipeline must process 36 million events in one hour: 10,000 events/sec average, but design for 2x headroom. Ingest supports 50k/sec, Spark supports 30k/sec, sink supports 12k/sec. Adding Spark executors cannot meet a 20k/sec target because sink is bottleneck. Options are sink batching/partitioning, a different serving design, a longer window or reduced write amplification.

Reasoning. Measure stages rather than guessing. Draw source -> ingest -> transform -> storage -> serving. For each, record capacity, observed throughput, latency, queue depth and limits. Apply simple capacity arithmetic: required throughput = daily/peak volume divided by useful processing window, adjusted for retries/headroom. Parallelism only helps when work can be partitioned and downstream accepts it.

Common mistake. A job runs slowly, so the team doubles compute. The source API is throttled at 500 requests/minute. Cloud cost doubles while duration barely changes.

Validation. A bottleneck table with measured/assumed capacities and the smallest change that raises end-to-end capacity.

Changed retry. Simulate losing one worker/partition domain and keep 25% headroom. Recalculate whether the design still meets the processing window.

6. DE17-06 | Reliability and failure domains

One acceptable solved answer. A daily pipeline has RTO 2 hours and RPO 0 for accepted raw files. Raw objects are immutable and replicated, transforms are replayable, curated publish is versioned, and last-known-good output remains available if validation fails. Compute can be recreated. The checkpoint state is backed up and tied to durable output so progress never outruns data.

Reasoning. Start from business impact, then choose RTO/RPO. Map state: raw source evidence, checkpoints, metadata, serving tables, secrets and code. Ask what happens if each domain disappears. Multi-zone compute without replicated state is false comfort. Replication also is not backup: corruption can replicate. Reliable data systems usually combine durable immutable inputs, idempotent/replayable processing, protected state/checkpoints, tested restore and controlled publish boundaries.

Common mistake. All services run across two zones, but credentials live in one manually maintained file and the only checkpoint is on one VM disk. The diagram looks redundant; actual recovery has single points of failure.

Validation. A failure-domain table: failure, blast radius, detection, safe state, recovery action, RTO/RPO evidence.

Changed retry. Tighten RTO from 4 hours to 15 minutes while keeping RPO zero. Identify which design changes cost more and which risks cannot be removed merely by adding replicas.

7. DE17-07 | Cost-aware architecture

One acceptable solved answer. Option A is serverless query at 5 currency units/TB, scanning 40 TB/day = 200/day. Partitioning/pruning cuts scans to 8 TB/day = 40/day. Option B uses always-on cluster costing 120/day but handles predictable heavy transformations. Compare workload shape and operations, then model 30-day cost and sensitivity at 2x volume.

Reasoning. Create a simple model before optimizing. Separate fixed and variable cost. Measure cost per business-relevant unit and by environment. Watch data movement: copying the same 20 TB between services can dominate cheap storage. Reliability has price - replicas and warm standby consume resources - but downtime also has business cost. The goal is to identify the cheapest architecture that satisfies the requirement set, not to delete reliability controls.

Common mistake. To save money a team removes duplicate raw retention and keeps only transformed outputs. A later bug cannot be replayed or audited. The immediate storage saving destroys recoverability and creates much larger incident cost.

Validation. A cost model showing assumptions, unit costs, volume, reliability overhead and a sensitivity case. Cost claims are traceable rather than 'serverless is cheaper'.

Changed retry. Double data volume and cut query frequency by half. Recompute rather than assuming your original winner stays cheapest.

8. DE17-08 | Security and governance by design

One acceptable solved answer. Orders arrive with email and payment token. Ingest identity can write raw but not read curated. Transform identity reads raw and writes curated. Analytics users read masked curated tables; only an approved support role can access restricted contact data. Secrets are retrieved from a managed secrets system; audit logs and lineage record access/change paths.

Reasoning. Draw trust boundaries across source, ingestion identity, raw storage, transform identity, curated serving and consumers. Prefer workload identity/short-lived credentials over embedded secrets where supported. Separate admin, producer and consumer privileges. Classify sensitive columns early so masking/tokenization and retention happen before broad analytics access. Governance also needs ownership, catalog/lineage, quality contracts and exception processes.

Common mistake. A pipeline fails with permission denied, so an engineer grants the service account subscription-owner access. The incident clears, but a small operational problem becomes a platform-wide security risk.

Validation. A security/governance matrix connecting persona -> data asset -> allowed action -> control -> evidence. No wildcard 'owner' role used as a shortcut.

Changed retry. Add a third-party data processor and cross-region copy. Identify new trust/compliance boundaries before changing service choices.

9. DE17-09 | Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes

One acceptable solved answer. An e-commerce company has PostgreSQL orders, a lakehouse, and many domain teams. Debezium-like CDC publishes order changes to Kafka; a data contract defines order-event keys, required fields and compatibility; workload identity/Vault supplies short-lived database credentials; the Orders domain owns the curated order data product; the platform team may run shared connectors/processors on managed Kubernetes. Each component has a different job. Removing one does not mean another automatically replaces it.

Reasoning. CDC is useful when incremental source changes matter and polling/full reloads are inefficient or too slow; it still requires ordering, schema evolution, deletes and replay thinking. Data contracts make interface expectations reviewable before producers silently break consumers. Vault-type systems centralize secret lifecycle and identity/policy instead of scattering static credentials. Data mesh is mainly an operating/ownership model, not a database product. Kubernetes can standardize container scheduling, service discovery and desired-state operations but adds platform complexity; data engineers should not deploy it merely because a pipeline uses containers.

Common mistake. A team says 'we are doing data mesh, so we need Kubernetes and Kafka.' That confuses organizational/data-product principles with implementation choices. Mesh can use many technologies; Kubernetes does not create domain ownership or trustworthy data products.

Validation. A five-column map: technology/concept, problem solved, where it sits, major risk/ownership, and when not to use it. You can explain each in 60 seconds without pretending production mastery.

Changed retry. Remove real-time requirements and move to daily file exchange with a small team. Re-evaluate which five concepts remain useful and which would be unjustified complexity.

10. DE17-10 | Architecture review and defense

One acceptable solved answer. Defend a regional retail lakehouse: hourly batch ingestion, immutable raw, Delta silver/gold, warehouse/SQL serving, managed orchestration, least-privilege identities, freshness/reconciliation gates, last-known-good publish, cost budgets and replay. Reviewers challenge: why not streaming? why not only warehouse? what if region fails? what if volume 5x? The defense returns to the requirement table and quantified trade-offs.

Reasoning. Use a repeatable review order: 1 requirements/constraints, 2 data flow and grain, 3 batch/stream/storage choices, 4 failure/replay/state, 5 security/governance, 6 scaling/cost, 7 observability/operations, 8 alternatives/trade-offs, 9 rollout/migration, 10 evidence. Do not hide uncertainty. A strong answer says 'I would measure X before choosing Y' when the required fact is unknown.

Common mistake. The presenter answers every challenge by naming another service. The diagram grows but requirements, costs and ownership remain unclear. Complexity is being used to hide uncertainty instead of resolving it.

Validation. A one-page architecture brief, diagram, ADRs and 5-minute verbal defense where each major component has a requirement-based reason and a failure/recovery story.

Changed retry. Use the fresh Gate-B-style case with different latency, budget and compliance constraints. Defend a changed architecture without copying your first diagram.